kurvst Typst API

`kurvst` is a Typst package backed by `kurvst.wasm`, a small Kurbo-based plugin. Its public API is path-based: construct a path dictionary, pass that path into transforms, and draw the returned path.

It exposes Bezier utilities that are awkward or unavailable in native Typst:

- constructing cubic and Hobby paths,
- trimming paths by arc length,
- generating sampled path patterns,
- fitting Kurbo offset/parallel paths,
- converting returned path geometry to CeTZ drawing commands.

Mark specifications

`kurvst.mark` provides named-only `triangle`, `straight`, `stealth`, `round`, `tikz`, `barb`, `hooks`, `bar`, `bracket`, `circle`, `square`, `diamond`, and `rays` constructors, plus positional `combine`. They return plain dictionaries tagged `kind: "kurvst-mark"`; they do not draw anything. Linnest and the native FeynKit renderer consume these specifications through the same geometry engine; stock CeTZ only paints the Typst route's finished geometry.

Dimensions `length` and `width` accept fixed lengths, ratios of line thickness, or mixed values such as `3pt + 450%`. `inset` and `shorten` are ratios; `shorten: 100%` is supported. `arc` and `phase` are angles, `rev` is boolean, `n` is a positive integer, `align` is `"center"` or `"end"`, and `fit` is `"chord"` (the engine default) or `"bend"`. Constructors omit defaults so the shared engine owns geometry.

Each head is built with its tip on the marked point and the line arriving from behind. Its paint keeps the nominal `length` along the line and `width` across it whether or not it is stroked: a stroke runs half its width inside the outline, and every mitred corner keeps its point however sharp. A head too small to hold its stroke that way grows just enough to hold it, keeping its tip (a centred shape its centre) in place: a `triangle`, `stealth`, `square`, or `diamond` keeps its shape, and the other shapes grow only the dimension that is too small. A `triangle` or `stealth` without area (zero length or width, or a 100% inset) cannot hold a stroke and is stroked as given. The shaft stops where the head's drawn path (its outline, or the stroke's centre line when stroked) meets the line at the rear; that distance is the mark's end, and a composite places each part at the previous part's end.

- `triangle`, `stealth`: a dart whose rear notch is `inset` of the length deep (defaults 0% and 40%). A stroked outline runs half a stroke inside the dart, so the mitred paint is exactly the dart.
- `round`: a dart with round joins. Its outline sits half a stroke inside every side of the box, so the rounded paint still touches the tip and the box, and its notch is `inset` (default 40%) of that inner dart's length.
- `straight`: an open chevron with round caps. Each arm's outer edge is the tangent from the tip to its end cap, so the mitred point lands on the tip.
- `tikz`: two quarter circles that leave the tip along the line in a cusp and curl out to the full width.
- `barb`: an arc of the full width bulging toward the tip, swept `arc` (default 180deg) symmetrically about the line. Complete positive or negative turns stop the shaft at the rear axis crossing; an open neighboring turn retains its gap and front crossing.
- `hooks`: two hooks, each half the width, that leave one root on the line toward the tip and curl away from it by `arc` (default 180deg).
- `bar`: a stroke across the line. `bracket`: a spine across the line at the front with arms running `length` back.
- `circle`, `square`, `diamond`: centred on the tip, or ending at it with `align: "end"`; `bar` aligns the same way.
- `rays`: `n` spokes of `length` from a hub on the tip, or one `length` behind it with `align: "end"`, the first at `phase`. By default no spoke runs along the line, where the shaft would hide it: an even count starts at 180deg / n and is symmetric about the line, and an odd count starts at −90deg, across the line, and is symmetric about that perpendicular.

`rev` turns a head end for end within its own length. Defaults scale with the line thickness: pointed heads (`triangle`, `stealth`, `round`, `straight`) are `3pt + 450%` long and 80% as wide; `tikz`, `barb`, and `hooks` are `3pt + 450%` wide; `bar` and `bracket` are `2.4pt + 360%` wide, a bracket 30% as long; `circle` and `square` are 400% wide and long, the `diamond` is that square turned by 45deg; `rays` are 280% long, four by default. Unnamed `triangle` and `stealth` heads are filled without a stroke; naming `fill` or `stroke` selects the general defaults, which fill closed shapes and stroke every shape.

Available shape fields:

- triangle, stealth, round: length, width, inset, fill, stroke, rev;
- straight: length, width, stroke, rev;
- tikz: width, stroke (deliberately no arc);
- barb, hooks: width, arc, stroke, rev;
- bar: width, stroke, align;
- bracket: length, width, stroke, rev;
- circle, square, diamond: length, width, fill, stroke, align;
- rays: length, n, phase, stroke, align.

Each shape also accepts fit and shorten. `combine(mark.bar(), 2pt, mark.bar(), fit: "bend")` accepts positional nested mark dictionaries and fixed/ratio/mixed gaps, with one fit and shorten on the composite; children must not set either. CeTZ symbol, scale, anchor, and shorten-to options are rejected rather than silently ignored.

Explicit auto width, rays phase, and bracket length are omitted so Rust applies defaults. Fill and stroke accept paint, none, auto, or booleans, not content, callbacks, or arbitrary dictionaries. Mark stroke thickness must remain auto: set line thickness on the line, not independently on the mark.

`kurvst.mark.prepare(spec)` returns CBOR-safe engine data: it removes kind, converts dimensions to `(points: ..., ratio: ...)`, ratios to fractions, and angles to radians. Composite parts become recursive specs or `(gap: (points: ..., ratio: ...))`. Context belongs to each batch template, not to preparation. It has positive units-per-pt and finite nonnegative line-thickness in drawing units; Rust resolves sizes as `points * units_per_pt + ratio * line_thickness`. Head stroke objects are paint-only: explicit thickness, cap, join, dash, and miter-limit modifiers are rejected rather than silently discarded.

`kurvst.mark.geometry(templates, carriers, placements, mode: "candidates")` runs one native batch. Each template has `mark` and `context` fields; carriers are Kurvst paths, and placements refer to template/carrier indices with start/end/ratio/distance stations, signed shifts, and forward/backward direction. Ratio and distance stations locate the painted longitudinal center: the midpoint of the local head outline's longitudinal bounds, including enabled stroke and all composite parts (not the shaft). They always use chord fit, including ratio zero and one. Start and end instead locate the template origin/contact and retain the requested endpoint fit, including bend. The engine applies the direction-aware center offset and signed user shift before clamping the origin inward to the visible carrier. A shift follows carrier direction even for backward marks. Short carriers never rescale a head; clamping may therefore prevent its painted center reaching the station. Native and Typst callers use this same rule without caller-side centering. Candidate mode returns independent candidate geometry. In `"selected"` mode, paint the returned head-only `marks` plus one shared `shafts` entry per carrier, rather than repainting an independent shaft beneath each selected head. The default `format: "native"` keeps the full native dictionaries unchanged. `format: "cbor"` returns encoded bytes for a packed envelope: decode that envelope once with `cbor(...)` to read indices, contacts, styles, and boxes. Only path-valued fields (`path`, `outline`, `shaft`, `shaft-outline`, and `footprint`) are independently encoded native-path CBOR byte strings. Kurvst's path readers (`elements`, `points`, `segments`, `to-cetz-data`, and `to-native`) accept these packets and decode only the path being consumed. Bounds-only consumers need not materialize any coordinates. Neither format changes fitting, candidate/selected semantics, or the number of batches. The Rust/Wasm `mark_geometry_packed_bytes`/`mark_geometry_packed` companion exports produce this packed envelope; `mark_geometry_bytes`/`mark_geometry` retain the full native DTO. These are distinct representations, not aliases or alternate geometry engines. Each head path includes `outline-bounds`, and each mark includes `footprint-bounds`: ordered conservative boxes with `left`, `right`, `bottom`, and `top` coordinates, one per nonempty subpath. Moves start new regions; endpoint and control coordinates contribute, including degenerate regions. These are control hulls of the actual transformed geometry, not tight curve extrema or transformed local rectangles. Region order and holes are retained. Use head `outline-bounds` when shaft paint is absent, even if sizing thickness is nonzero; otherwise use `footprint-bounds`. A separately filled or patterned shaft still requires its own bounds. Selected footprints remain head-only. Paint becomes boolean enabled flags (`none` disables); auto is omitted so shared defaults apply. Actual paint remains in the original dictionary for the caller's drawing style, never in the numeric payload.

Choose an import path

Kurvst is currently a bundled source package, not a Typst Universe package. A Clinnet run writes it below `build/templates/crates/kurvst/typst/`; a custom template in `build/templates/` can therefore import it with:

```
#import "crates/kurvst/typst/src/lib.typ" as kurvst
```

Repository examples live one directory beside `src/` and instead use `#import "../src/lib.typ" as kurvst`. Keep `kurvst.wasm`, `typst.toml`, and the `src/` tree together when copying the package to another Typst project.

Path geometry points are two-item tuples:

```
#let p = (1.2, -0.4)
```

Cubic segments use `start`, `control-start`, `control-end`, and `end`:

```
#let segment = (
  start: (0, 0),
  control-start: (1, 0.5),
  control-end: (2, -0.5),
  end: (3, 0),
)
```

Use `from-cubic(segment)` once to turn that cubic dictionary into a path dictionary. Returned paths contain a curve-command wire path in `path`. Rust converts that wire path to Kurbo's `BezPath` internally, and every path-level Kurvst function accepts either a returned dictionary or the raw wire path.

```
#let path = kurvst.from-cubic(segment)
#let trimmed = kurvst.trim(path, start-outset: 0.2, end-outset: 0.1)
#let shifted = kurvst.parallel(trimmed, distance: 0.15)
```

Path Wire Format

A Kurvst wire path is a dictionary with one `elements` array. The elements mirror Typst's native `curve` commands, but points are plain numeric tuples and closure is always straight:

```
#let path = (
  elements: (
    (kind: "move", start: (0, 0)),
    (kind: "line", end: (0.6, 0.3)),
    (kind: "quad", control: (1.0, 1.0), end: (1.4, 0.3)),
    (
      kind: "cubic",
      control-start: (1.8, -0.4),
      control-end: (2.4, 1.0),
      end: (3.0, 0),
    ),
    (kind: "close"),
  ),
)
```

The supported element kinds are:

- `move`: starts a subpath at `start`.
- `line`: adds a straight segment ending at `end`.
- `quad`: adds a quadratic segment using `control` and `end`.
- `cubic`: adds a cubic segment using `control-start`, `control-end`, and `end`.
- `close`: closes the current subpath with a straight segment back to its start. It has no `mode` argument or field. To close along a curve, add explicit curve segments ending at the subpath start before `close()`. This keeps measurements, transforms, and native/CeTZ drawing on the same geometry.

Unknown element fields are rejected by both the Typst constructors/emitters and the Rust-backed geometry operations.

Prefer the path fragment helpers for new code. `line`, `quad`, and `cubic` include their start points, so fragments can be built independently:

```
#let custom = kurvst.path(
  kurvst.line((0, 0), (0.6, 0.3)),
  kurvst.quad((0.6, 0.3), (1.0, 1.0), (1.4, 0.3)),
  kurvst.cubic((1.4, 0.3), (1.8, -0.4), (2.4, 1.0), (3.0, 0)),
)
```

`arc(center, radius, start, stop)` builds a circular arc from cubic pieces of at most 90°. Angles are measured counterclockwise from the positive x axis in y-up coordinates, as in CeTZ; `stop < start` runs clockwise:

```
#let stadium = kurvst.path(
  kurvst.arc((1, 0.5), 1, 0deg, 180deg),
  kurvst.line((0, 0.5), (0, -0.5)),
  kurvst.arc((1, -0.5), 1, 180deg, 360deg),
  kurvst.close(),
)
```

`path` and `append` flatten fragments. If an appended fragment starts where the current path ends, its leading `move` is skipped; if it starts elsewhere, the `move` begins a new subpath. The edited path can go straight back into Kurvst transforms:

```
#let base = kurvst.from-cubic(segment)
#let extended = kurvst.append(base, kurvst.line(segment.end, (3.4, 0.2)))

#let trimmed = kurvst.trim(extended, start-outset: 0.15)
#let shifted = kurvst.parallel(trimmed, distance: 0.1)

#kurvst.to-native(shifted, unit: 36pt, stroke: rgb("#1b7f4c") + 0.7pt)
```

Use `elements` when you need the raw command stream, `points` for the visited endpoints, `segments` for cubic segment dictionaries, `to-native` for native Typst drawing, and `to-cetz-data` or `to-cetz` for CeTZ.

`outset-point(from, toward, distance:)` moves along the line between two points. Positive distances are capped at their separation, so a one-ended outset can reach a large node boundary without passing `toward`. Negative distances move away without a cap; coincident points return `from`. For example, moving `(0, 0)` toward `(10, 0)` by `8` gives `(8, 0)`, requesting `12` gives `(10, 0)`, and requesting `-2` gives `(-2, 0)`.

A typical chain keeps the returned dictionaries intact until the final drawing step:

```
#let base = kurvst.hobby-through(demo-start, demo-through, demo-end)
#let trimmed = kurvst.trim(base, start-outset: 0.2, end-outset: 0.1)
#let shifted = kurvst.parallel(trimmed, distance: 0.15)

#cetz.canvas({
  kurvst.to-cetz(base, stroke: rgb("#bbbbbb") + 0.4pt)
  kurvst.to-cetz(trimmed, stroke: rgb("#d72638") + 0.6pt)
  kurvst.to-cetz(shifted, stroke: rgb("#1b7f4c") + 0.6pt)
})
```

Paths And Trimming

`cubic` constructs a path from one cubic Bezier. `trim` trims by curve distance from the start and/or end and returns another path dictionary.

```
#let path = kurvst.from-cubic(segment)
#let trimmed = kurvst.trim(path, start-outset: 0.15, end-outset: 0.1)
```

`intersections(a, b)` returns transverse crossings sorted by arc distance along `a`. Each record includes `point`, `distance-a`, `distance-b`, segment indices, and local segment parameters. Path-endpoint and tangential contacts are omitted, so the distances can be passed directly to path-splitting logic.

`hobby-through(start, through, end)` builds a smooth open curve through three points; `segments` returns the two cubic halves split at `through`. `hobby-spline(points)` does the same for any open point sequence with at least two points. Both return path dictionaries and can be passed directly to path-level helpers.

```
#let spline = kurvst.hobby-spline((
  (0.0, 0.0),
  (0.9, 0.8),
  (1.8, -0.3),
  (2.8, 0.4),
))
#let shifted = kurvst.parallel(spline, distance: 0.14)
#let wiggle = kurvst.pattern(
```

```
  spline,
  pattern: kurvst.wave(samples-per-period: 24),
  amplitude: 0.1,
  wavelength: 0.55,
)
#cetz.canvas({
  kurvst.to-cetz(spline, stroke: rgb("#bbbbbb") + 0.45pt)
  kurvst.to-cetz(shifted, stroke: rgb("#1b7f4c") + 0.6pt)
  kurvst.to-cetz(wiggle, stroke: rgb("#355c9a") + 0.55pt)
})
```

`split-through` combines Hobby spline construction with endpoint trimming and returns one path part between each consecutive point. Consumers such as Linnest can wrap it for graph-edge geometry.

Path Patterns

`pattern` generates a one-dimensional pattern along any path dictionary. Built-ins are regular Typst dictionaries:

```
#let wave = kurvst.wave()
#let zigzag = kurvst.zigzag()
#let coil = kurvst.coil(longitudinal-scale: 1.6)
```

String names `"wave"`, `"zigzag"`, and `"coil"` are accepted for convenience, but they are resolved in Typst before calling wasm.

`split-at` takes arc distances along the input path and returns the one patterned path cut there as `parts`, so pieces can be styled independently while following a single pattern:

```
#let base = kurvst.from-cubic(segment)
#let split = kurvst.pattern(base, pattern: "coil", amplitude: 0.12,
  wavelength: 0.4, split-at: (kurvst.length(base) / 2,))
#cetz.canvas({
  kurvst.to-cetz(split.parts.at(0), stroke: blue + 0.6pt)
  kurvst.to-cetz(split.parts.at(1), stroke: orange + 0.6pt)
})
```

Cuts must be finite. They are sorted and clamped to `[0, L]`; duplicate cuts and cuts at the endpoints retain empty parts, so there are always one more parts than cuts. With no cuts, the only part is the full pattern. A zero-length carrier produces empty parts. The full `path` is unchanged: sampling, phase, endpoint tapering, and spline fitting use the full carrier once. Parts are exact subdivisions, with distances mapped linearly between carrier samples rather than measured along the longer decorated curve.

Custom patterns use the same object shape:

```
#let hook = (
  kind: "points",
  name: "hook",
  interpolation: "linear",
  points: (
    (at: 0, x: 0, y: 0),
    (at: 0.25, x: 0.15, y: 1),
    (at: 0.5, x: 0, y: 0),
    (at: 0.75, x: -0.15, y: -1),
    (at: 1, x: 0, y: 0),
  ),
)
```

`at` is the phase position inside one wavelength, from `0` to `1`. The `x` coordinate offsets along the path tangent and `y` offsets along the path normal; both are scaled by `amplitude`. If `at` is omitted, points are spaced evenly. Use `interpolation: "linear"` for corners and `"smooth"` for a spline through sampled points.

`kurvst.coil` defaults to `samples-per-period: 16`, `longitudinal-scale: 1.25`, `fit-length: none`, `amplitude: 0.1`, and `wavelength: 1.0`. With `fit-length: none`, the periodic coil is unchanged; `amplitude` and `wavelength` are used only for fitting.

Set `fit-length: L` to return an ordinary normalized point-pattern dictionary spanning `P = max(1, round(L / wavelength)) - 0.5` coil periods, using the requested wavelength. It retains full amplitude with inward endpoint phases, exact zero offsets at both ends, and `endpoint-ramp: false`: no taper, straight stubs, or connectors. Longitudinal fitting depends on `amplitude`, so apply the dictionary once over a carrier of length `L` with the same amplitude, a wavelength of `L`, all its samples, and zero phase:

```
#let base = kurvst.from-cubic(segment)
#let L = kurvst.length(base)
#let A = 0.12
#let coil = kurvst.coil(fit-length: L, amplitude: A, wavelength: 0.55)
#let fitted = kurvst.pattern(
  base,
  pattern: coil,
  amplitude: A,
  wavelength: L,
  samples-per-period: coil.points.len() - 1,
  phase: 0,
)
```

Fitting requires positive, finite `fit-length` and `wavelength`, finite `amplitude`, and finite, nonnegative `longitudinal-scale`. Kurvst's numeric engine owns fitting and sampling. To paint a fitted coil without transferring its points through Typst, pass a dictionary with `kind: "fitted-coil"`, `fit-length: L`, and the same fitting parameters directly as `pattern`. Keep the outer `amplitude: A`, `wavelength: L`, and `phase: 0`; the engine selects the fitted sample count. The public `kurvst.coil(...).points` uses that same sampler. As for other point patterns, curved carriers use local tangent/normal offsets, not evaluation at a corrected arc distance. `endpoint-slope` has no effect with `endpoint-ramp: false`.

Patterns with `endpoint-ramp: true`, including periodic coils, taper amplitude over 75% of a wavelength at each anchored endpoint, capped at half the path length for short paths. This is useful for coils, whose longitudinal offset would otherwise put the first and last visible points inside the turn near nodes. The longitudinal offset uses the square of the taper envelope, letting the coil open sideways before it starts doubling back.

On `kurvst.pattern`, `endpoint-slope: 0` keeps the taper tangential to the base path. Values from `0` to `3` control the envelope's initial slope; try `1` for an angled finish with coil `phase: calc.pi / 2`. This keeps the endpoint attached and still flattens the envelope into the full-sized coil. It is not an angle: the direction also depends on amplitude, wavelength, and the lateral offset at the endpoint phase. Unanchored endpoints and patterns without `endpoint-ramp` are unaffected.

```
#let base = kurvst.from-cubic(segment)
#let path = kurvst.pattern(
  base,
  pattern: hook,
  amplitude: 0.15,
  wavelength: 0.7,
)
#native-scene({
  native-cubics(kurvst.segments(base), stroke: rgb("#c8c8c8") + 0.45pt)
  native-polyline(kurvst.points(path), stroke: rgb("#1b7f4c") + 0.8pt)
})
```

Parallel Paths

`parallel` uses Kurbo's offset curve fitter to produce a path at a fixed normal distance from the source path. Positive distances follow the left normal of the path direction; negative distances follow the right normal. Within each connected source subpath, differing fitted endpoints are joined by straight bevel segments, including at sharp corners and the closing seam. Fitted controls are not snapped; bevels need not stay at the fixed normal distance. Explicit moves remain separate subpaths even when their coordinates coincide. Closed subpaths retain closure unless trimming cuts them open. `start-outset` and `end-outset` trim the fitted path by arc length after the offset and bevels are computed, across all subpaths without counting move gaps.

```
#let base = kurvst.from-cubic(segment)
#let left = kurvst.parallel(base, distance: 0.18, start-outset: 0.35, end-outset: 0.35)
#let right = kurvst.parallel(base, distance: -0.18)
```

```
#native-scene({
  native-cubics(kurvst.segments(base), stroke: rgb("#c8c8c8") + 0.45pt)
  native-cubics(kurvst.segments(left), stroke: rgb("#1b7f4c") + 0.8pt)
  native-cubics(kurvst.segments(right), stroke: rgb("#355c9a") + 0.8pt)
})
```

Stroke Outlines

`outline` turns a stroked path into a closed region, so a line drawing becomes a filled solid with the same visible shape. It wraps Kurbo's stroker: `join` is `"miter"`, `"round"`, or `"bevel"`, and `cap` (or `start-cap` and `end-cap` separately) is `"butt"`, `"square"`, or `"round"`. Open subpaths become one contour; closed subpaths become an outer and an inner contour, so fill with the default non-zero rule.

```
#let base = kurvst.path(
  kurvst.line((0, 1), (0, 0)),
  kurvst.cubic((0, 0), (1, 0), (2, 1), (3, 0)),
)
#let solid = kurvst.outline(base, width: 0.2, cap: "round")
#cetz.canvas(kurvst.to-cetz(solid, fill: black, stroke: none))
```

CeTZ `merge-path` bridges separate subpaths with straight lines. That leaves fills unchanged, but a stroked outline with several contours shows the bridge; use `to-native` to stroke the contours themselves. See `examples/outline-logo.typ` for a full logo converted to solids.

Path Layers

`layer` is a convenience wrapper for drawing derived visible paths. It combines side-aware offsetting, endpoint outsets, and shortening into one path-in/path-out operation. Offsetting happens first: `length` is a fixed visible arc length, `ratio` is a fraction of the full offset path length before trimming, and `resolve-length` decides how to combine them. The default `"min"` keeps whichever limit is shorter. `start-outset`, `end-outset`, and `shift` are arc distances on the offset path. Positive `shift` moves the shortened interval toward the path end; it is clamped to preserve the endpoint outsets.

`resolve-length` and the `method` argument of `resolve-length(...)` accept `"min"`/`"shorter"`, `"max"`/`"longer"`, `"length"`/`"fixed"`, `"ratio"`/`"relative"`, and `"none"`/`"full"`. Non-positive limits are treated as absent. Except for `"none"`/`"full"`, a sole provided limit is used regardless of the policy; with two limits, `"length"` prefers the fixed length and `"ratio"` the relative length. `"none"`/`"full"` always keeps the full path, ignoring both limits. Unknown policy strings are rejected.

A custom resolver receives one dictionary with `base-length`, `length`, and `ratio`. The latter two entries are positive absolute lengths or `none`: `ratio` has already been multiplied by `base-length`. Return an absolute target length, or `none` to keep the full path.

Empty layers remain empty, with or without an offset; invalid resolver names are still rejected.

```
#let base = kurvst.hobby-spline((
  (0.0, 0.0),
  (0.9, 0.8),
  (1.8, -0.3),
  (2.8, 0.4),
))
#let layer = kurvst.layer(
  base,
  offset: 0.16,
  length: 1.6,
  ratio: 0.5,
  resolve-length: "min",
  shift: 0.35,
)
#cetz.canvas({
  kurvst.to-cetz(base, stroke: rgb("#bbbbbb") + 0.45pt)
  kurvst.to-cetz(layer, stroke: rgb("#1b7f4c") + 0.8pt)
})
```

Use `side-point` to choose the sign of the offset from a point on the desired side of the original path, before offsetting or trimming. Drawing packages can use this to place derived layers on the same side as an edge label without knowing anything about the physics or graph style that requested the layer. The returned `offset` records the resolved signed distance, including for empty paths. When processing fragments of the same carrier, reuse this value with `side-point: none` so all fragments use the full carrier's side choice.

`layers(path, shifts, ..options)` prepares that carrier once and returns a `(path: ..., segments: ...)` record for each shift. The geometry options, resolver policies, and callback dictionary are the same as for `layer`. `"none"` and `"full"` ignore both length limits. Unknown policies and unsupported options are rejected even on empty paths or empty batches. `format: "cbor"` returns encoded layers and footprints plus `count`, `nonzero`, `supported`, `all-single`, and the shared `offset`. The encoded paths omit that offset metadata. `unit` scales multi-segment footprints only; a single cubic retains unit 1.

`frames(path, distances)` samples points and unnormalized tangents using the same prefix trimming as `trim`. Distances are clamped to the path; empty paths return `none` for each distance. `format: "cbor"` retains encoded records for transfer to native drawing operations.

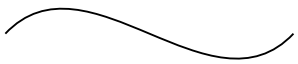
`region-samples(parts, regions: 4, unit: 1, step: 4, accuracy: 0.001)` samples overlapping points in equal arc-length regions of cubic parts with `segments` and `visible` fields. Hidden parts contribute length but no output. Both endpoints of every trimmed cubic are retained, including shared endpoints; `step` bounds control-polygon sampling distance after scaling. Sampling density uses the magnitude of `unit`, so negative and positive scales give the same samples. Returned point coordinates stay unscaled.

`pattern-to-cetz(path, unit: 1, style: (:), ..options)` generates and draws a pattern directly. Geometry options match `pattern`; CeTZ style stays in the separate `style` dictionary. Numeric coordinates can be transferred directly from the geometry engine without an intermediate wire path.

Native Drawing Primitives

Kurvst returns dictionaries and arrays. The core geometry can be drawn without CeTZ by emitting native `curve` content:

```
#kurvst.to-native(kurvst.from-cubic(segment), unit:
36pt, stroke: black + 0.7pt)
```



The generated reference below keeps drawing code inline. It only uses the small common helpers above for coordinate scaling, native `curve` construction, point markers, and fixed-size preview blocks.

CeTZ Interoperability

The CeTZ helpers are thin adapters over the same returned geometry. Use them when the surrounding document already lives in a CeTZ canvas, or when you want CeTZ path merging and styling:

```
#cetz.canvas({
  let base = kurvst.from-cubic(segment)
  kurvst.to-cetz(base, stroke: rgb("#d72638") +
0.6pt)

  let path = kurvst.pattern(
    base,
    pattern: kurvst.coil(longitudinal-scale: 1.6),
    amplitude: 0.12,
    wavelength: 0.55,
  )
  kurvst.to-cetz(path, stroke: rgb("#355c9a") +
0.55pt)
})
```



Generated Reference

kurvst

- append()
- arc()
- center-outset()
- close()
- coil()
- cubic()
- cubic-point()
- cubic-tangent()
- cubic-to()
- elements()
- frames()
- from-cubic()
- from-elements()
- hobby-spline()
- hobby-through()
- intersections()
- layer()
- layers()
- length()
- line()
- line-segment()
- line-to()
- move-to()
- outline()
- outset-point()
- parallel()
- path()
- pattern()
- pattern-to-cetz()
- point()
- points()
- quad()
- quad-to()
- region-samples()
- resolve-length()
- segments()
- split-through()
- to-cetz()
- to-cetz-data()
- to-native()
- trim()
- wave()
- zigzag()

Variables

- layer-defaults

append

Return a path with additional fragments or elements appended.

Parameters

```
append(
  path: dictionary,
  ..parts: any
) -> dictionary
```

path `dictionary`

Base Kurvst path dictionary.

..parts `any`

Path fragments, path elements, or element arrays to append.

arc

Build a circular arc path fragment from cubic segments.

Parameters

```
arc(
  center: array,
  radius: int float,
  start: angle,
  stop: angle
) -> dictionary
```

center `array`

Arc center.

radius `int` or `float`

Positive arc radius.

start `angle`

Start angle, measured counterclockwise from the positive x axis.

stop `angle`

Stop angle; `stop < start` runs clockwise.

center-outset

Compute the symmetric trim needed to center a shorter path layer.

Parameters

```
center-outset(
  base-length: int float,
  length: none int float,
  ratio: none int float,
  resolve-length: string function,
  start-outset: int float,
  end-outset: int float
) -> int float
```

base-length `int` or `float`

Full base path arc length.

length `none` or `int` or `float`

Fixed target visible length.

Default: `none`

ratio `none` or `int` or `float`

Relative target visible length as a fraction of `base-length`.

Default: `none`

resolve-length `string` or `function`

Resolution strategy for fixed and relative targets.

Default: `"min"`

start-outset `int` or `float`

Already-applied trim at the start of the path.

Default: `0`

end-outset `int` or `float`

Already-applied trim at the end of the path.

Default: `0`

close

Close the current subpath with a straight segment back to its start.

For curved closure, add explicit curve segments before calling `close`.

Parameters

```
close() -> dictionary
```

coil

A smooth coil path pattern.

Parameters

```
coil(
  samples-per-period: int,
  longitudinal-scale: int float,
  fit-length: none int float,
  amplitude: int float,
  wavelength: int float
) -> dictionary
```

samples-per-period `int`

Number of samples used to approximate one coil period.

Default: `16`

longitudinal-scale `int` or `float`

Horizontal scale of the coil before it is mapped onto a path.

Default: `1.25`

fit-length `none` or `int` or `float`

Fit a complete coil between inward-facing natural endpoints, without tapering. A positive path length fits half-integer periods using wavelength as nominal spacing. Apply once with pattern wavelength equal to fit-length, phase zero, and samples-per-period equal to the returned points.len() - 1.

Default: `none`

amplitude `int` or `float`

Amplitude used for longitudinal fitting; pass the same amplitude to pattern.

Default: `0.1`

wavelength `int` or `float`

Requested coil spacing when fit-length is set.

Default: `1.0`

cubic

Build a cubic path fragment.

Parameters

```
cubic(
  start: array,
  control-start: array,
  control-end: array,
  end: array
) -> dictionary
```

start `array`

Start point.

control-start `array`

Cubic control point near `start`.

control-end `array`

Cubic control point near `end`.

end `array`

Endpoint.

cubic-point

Evaluate a cubic segment at parameter `t`.

Parameters

```
cubic-point(
  segment: dictionary,
  t: int float
) -> array
```

segment `dictionary`

Segment with `start`, `control-start`, `control-end`, and `end`.

t `int` or `float`

Segment parameter in the range `[0, 1]`.

cubic-tangent

Evaluate the tangent of a cubic segment at parameter `t`.

Parameters

```
cubic-tangent(
  segment: dictionary,
  t: int float
) -> array
```

segment `dictionary`

Segment with `start`, `control-start`, `control-end`, and `end`.

t `int` or `float`

Segment parameter in the range `[0, 1]`.

cubic-to

Build a `cubic` path element.

Parameters

```
cubic-to(
  control-start: array,
  control-end: array,
  end: array
) -> dictionary
```

control-start `array`

Cubic control point near the start point.

control-end `array`

Cubic control point near the endpoint.

end `array`

Cubic endpoint.

elements

Return the command elements that make up a Kurvst path.

Parameters

```
elements(path: dictionary) -> array
```

path `dictionary`

Kurvst path dictionary to inspect.

frames

Sample points and tangent directions at arc distances, clamped to the path. Uses the same prefix trimming as trim; empty paths return none per sample. Tangents are not normalized. `format: "cbor"` retains the encoded records for transport to another native geometry operation.

Parameters

```
frames(
  path,
  distances,
  accuracy,
  format
) -> array bytes
```

from-cubic

Build a path fragment from a cubic segment dictionary.

Parameters

```
from-cubic(segment: dictionary) -> dictionary
```

segment `dictionary`

Segment with `start`, `control-start`, `control-end`, and `end`.

from-elements

Build a path dictionary from an existing element array.

Parameters

```
from-elements(elements: array) -> dictionary
```

elements `array`

Array of Kurvst path elements.

hobby-spline

Construct a Hobby spline through an arbitrary point sequence.

Parameters

```
hobby-spline(
  points: array,
  omega: float,
  accuracy: float
) -> dictionary
```

points `array`

Two or more points for the open spline to pass through.

omega `float`

Hobby curl/tension parameter.

Default: `1.0`

accuracy `float`

Geometry approximation accuracy passed to the Rust geometry engine.

Default: `0.001`

hobby-through

Construct a cubic Hobby path through three points.

Parameters

```
hobby-through(
  start: array,
  through: array,
  end: array,
  omega: float,
  accuracy: float
) -> dictionary
```

start `array`

Start point.

through `array`

Intermediate point that the curve passes through.

end `array`

Endpoint.

omega `float`

Hobby curl/tension parameter.

Default: `1.0`

accuracy `float`

Geometry approximation accuracy passed to the Rust geometry engine.

Default: `0.001`

intersections

Find transverse crossings between two paths, sorted by arc distance along the first path.

Parameters

```
intersections(
  a: dictionary,
  b: dictionary,
  accuracy: float
) -> array
```

a `dictionary`

First Kurvst path dictionary.

b `dictionary`

Second Kurvst path dictionary.

accuracy `float`

Absolute geometry and arc-length tolerance.

Default: `0.001`

layer

Build a derived visible path layer. Returns drawable geometry and `offset`, the resolved signed offset before trimming.

Parameters

```
layer(
  path: dictionary,
  offset: int float,
  length: none int float,
  ratio: none int float,
  resolve-length: string function,
  shift: int float,
  start-outset: int float,
  end-outset: int float,
  side-point: none array,
  accuracy: float
) -> dictionary
```

path `dictionary`

Base Kurvst path dictionary.

offset `int` or `float`

Signed normal offset distance.

Default: `0`

length `none` or `int` or `float`

Fixed target visible length.

Default: `none`

ratio `none` or `int` or `float`

Relative target visible length as a fraction of the full offset path length.

Default: `none`

resolve-length `string` or `function`

Resolution strategy for fixed and relative targets.

Default: `"min"`

shift `int` or `float`

Arc-length displacement on the offset path; positive moves toward its end.

Default: `0`

start-outset `int` or `float`

Arc length removed from the start.

Default: `0`

end-outset `int` or `float`

Arc length removed from the end.

Default: `0`

side-point `none` or `array`

Optional point used to choose the sign of `offset`.

Default: `none`

accuracy `float`

Geometry approximation accuracy passed to the Rust geometry engine.

Default: `0.001`

layers

Build several shifted layers with one carrier preparation. Each result contains the normal layer `path` dictionary and its drawable cubic `segments`. Length resolution and offsetting match `layer`, including resolver validation for empty paths and empty batches. `format: "cbor"` returns encoded native layers and footprint paths with count, nonzero, supported, all-single and shared offset metadata. Native layer paths omit the shared offset field. `unit` affects footprint geometry only: single-segment Beziers retain unit 1, matching the drawing owner.

Parameters

```
layers(
  path: dictionary,
  shifts: array,
  offset: int float,
  length: none int float,
  ratio: none int float,
  resolve-length: string function,
  start-outset: int float,
  end-outset: int float,
  side-point: none array,
  accuracy: float,
  format: string,
  unit: int float
) -> array dictionary
```

path `dictionary`

Base Kurvst path dictionary.

shifts `array`

Arc-length displacements, clamped as in `layer`.

offset `int` or `float`

Signed normal offset distance.

Default: `0`

length `none` or `int` or `float`

Fixed target visible length.

Default: `none`

ratio `none` or `int` or `float`

Relative target visible length as a fraction of the full offset path length.

Default: `none`

resolve-length `string` or `function`

Resolution policy accepted by resolve-length, or its dictionary callback.

Default: `"min"`

start-outset `int` or `float`

Arc length removed from the start.

Default: `0`

end-outset `int` or `float`

Arc length removed from the end.

Default: `0`

side-point `none` or `array`

Optional point choosing the offset sign.

Default: `none`

accuracy `float`

Geometry approximation accuracy.

Default: `0.001`

format `string`

Return decoded layers or an opaque geometry packet.

Default: `"array"`

unit `int` or `float`

Numeric scale for multi-segment footprint paths.

Default: `1`

length

Compute the arc length of a path dictionary.

Parameters

```
length(
  path: dictionary,
  accuracy: float
) -> int float
```

path `dictionary`

Kurvst path dictionary to measure.

accuracy `float`

Arc-length approximation accuracy passed to the Rust geometry engine.

Default: `0.001`

line

Build a straight-line path fragment.

Parameters

```
line(
  start: array,
  end: array
) -> dictionary
```

start `array`

Start point.

end `array`

Endpoint.

line-segment

Build a cubic segment dictionary for a straight line.

Parameters

```
line-segment(
  start: array,
  end: array
) -> dictionary
```

start `array`

Start point.

end `array`

Endpoint.

line-to

Build a `line` path element.

Parameters

```
line-to(end: array) -> dictionary
```

end `array`

Line endpoint.

move-to

Build a `move` path element.

Parameters

```
move-to(start: array) -> dictionary
```

start `array`

New current point and subpath start.

outline

Expand a stroked path into a closed outline that can be filled.

Parameters

```
outline(
  path: dictionary,
  width: int float,
  join: string,
  miter-limit: int float,
  cap: string,
  start-cap: auto string,
  end-cap: auto string,
  accuracy: float
) -> dictionary
```

path `dictionary`

Kurvst path dictionary to stroke.

width `int` or `float`

Stroke width in path units.

Default: `0.1`

join `string`

Corner join: `"miter"`, `"round"`, or `"bevel"`.

Default: `"miter"`

miter-limit `int` or `float`

Miter length limit as a multiple of the width; longer miters are beveled.

Default: `4`

cap `string`

End cap for both ends: `"butt"`, `"square"`, or `"round"`.

Default: `"butt"`

start-cap `auto` or `string`

Cap at each subpath start; `auto` uses `cap`.

Default: `auto`

end-cap `auto` or `string`

Cap at each subpath end; `auto` uses `cap`.

Default: `auto`

accuracy `float`

Curve fitting tolerance passed to the Rust geometry engine.

Default: `0.001`

outset-point
Move `from` along the line toward `toward`, without passing the target.

Positive distances are capped at the point separation. Negative distances move away without a cap. Coincident points return `from`.

Parameters

```
outset-point(
  from: array,
  toward: array,
  distance: int float
) -> array
```

from `array`

Point to move.

toward `array`

Target point that defines the direction.

distance `int` or `float`

Signed requested movement from `from` toward `toward`.

Default: `0`

parallel
Generate a parallel path for a path.

Parameters

```
parallel(
  path: dictionary,
  distance: int float,
  start-outset: int float,
  end-outset: int float,
  accuracy: float
) -> dictionary
```

path `dictionary`

Base Kurvst path dictionary.

distance `int` or `float`

Signed normal offset distance.

Default: `0`

start-outset `int` or `float`

Arc length removed from the start of the offset path.

Default: `0`

end-outset `int` or `float`

Arc length removed from the end of the offset path.

Default: `0`

accuracy `float`

Geometry approximation accuracy passed to the Rust geometry engine.

Default: `0.001`

path

Build a path dictionary from path fragments or elements.

Parameters

```
path(..parts: any) -> dictionary
```

..parts `any`

Path fragments, path elements, or element arrays to concatenate.

pattern

Apply a repeated path pattern to a base path.

Parameters

```
pattern(
  path: dictionary,
  pattern: string dictionary,
  amplitude: int float,
  wavelength: int float,
  phase: int float,
  samples-per-period: int,
  coil-longitudinal-scale: int float,
  anchor-start: bool,
  anchor-end: bool,
  endpoint-slope: int float,
  accuracy: float,
  split-at: array
) -> dictionary
```

path `dictionary`

Base Kurvst path dictionary.

pattern `string` or `dictionary`

Pattern dictionary or built-in pattern name.

Default: `"wave"`

amplitude `int` or `float`

Normal amplitude of the pattern in path units.

Default: `0.1`

wavelength `int` or `float`

Arc length of one pattern period.

Default: `1.0`

phase `int` or `float`

Initial phase offset in radians.

Default: `0`

samples-per-period `int`

Samples per period for string-resolved patterns and smooth point-pattern mapping.

Default: `16`

coil-longitudinal-scale `int` or `float`

Longitudinal scale used when resolving the built-in coil pattern.

Default: `1.25`

anchor-start `bool`

Force the generated path to start on the base path.

Default: `true`

anchor-end `bool`

Force the generated path to end on the base path.

Default: `true`

endpoint-slope `int` or `float`

Initial slope of an anchored taper envelope, from 0 to 3. Zero keeps tangential ends; positive values allow angled ends when the pattern's lateral offset is nonzero at its endpoint phase. This is not an angle. Ignored for unanchored ends and patterns without endpoint ramping, including fitted coils.

Default: `0`

accuracy `float`

Geometry approximation accuracy passed to the Rust geometry engine.

Default: `0.001`

split-at `array`

Arc distances along `path` at which to cut the result into `parts`, one patterned path split into independently stylable pieces. Cuts are sorted and clamped; repeated and endpoint cuts retain empty parts.

Default: `()`

pattern-to-cetz

Generate and draw a patterned path through CeTZ. Geometry options are the same as pattern; drawing styles are kept separate.

Parameters

```
pattern-to-cetz(
  path: dictionary,
  unit: int float length,
  style: dictionary,
  ..options: arguments
) -> array
```

path `dictionary`

Base Kurvst path dictionary.

unit `int` or `float` or `length`

Scale applied to the generated coordinates.

Default: 1

style `dictionary`

CeTZ drawing style.

Default: `(:)`

..options `arguments`

Geometry options accepted by pattern.

point

Build a numeric point tuple.

Parameters

```
point(
  x: int float,
  y: int float
) -> array
```

x `int` or `float`

X coordinate.

y `int` or `float`

Y coordinate.

points

Return the points visited by a Kurvst path's command stream.

Parameters

```
points(path: dictionary) -> array
```

path `dictionary`

Kurvst path dictionary to inspect.

quad

Build a quadratic path fragment.

Parameters

```
quad(
  start: array,
  control: array,
  end: array
) -> dictionary
```

start `array`

Start point.

control `array`

Quadratic control point.

end `array`

Endpoint.

quad-to

Build a `quad` path element.

Parameters

```
quad-to(
  control: array,
  end: array
) -> dictionary
```

control `array`

Quadratic control point.

end `array`

Quadratic endpoint.

region-samples

Sample overlapping points in equal arc-length regions across cubic parts. Hidden parts contribute length but produce no points. Both endpoints of every trimmed cubic are retained, including shared endpoints.

Parameters

```
region-samples(
  parts: array,
  regions: int,
  unit: int float,
  step: int float,
  accuracy: int float
) -> array
```

parts `array`

Parts with cubic `segments` and a boolean `visible` field.

regions `int`

Number of equal arc-length regions.

Default: `4`

unit `int` or `float`

Coordinate-scale magnitude used for sampling density; points stay unscaled.

Default: `1`

step `int` or `float`

Maximum control-polygon step after scaling.

Default: `4`

accuracy `int` or `float`

Positive arc-length tolerance.

Default: `0.001`

resolve-length

Resolve a fixed and relative visible path length.

Parameters

```
resolve-length(
  base-length: int float,
  length: none int float,
  ratio: none int float,
  method: string function
) -> none int float
```

base-length `int` or `float`

Full base path arc length.

length `none` or `int` or `float`

Fixed target arc length.

Default: `none`

ratio `none` or `int` or `float`

Relative target length as a fraction of `base-length`.

Default: `none`

method `string` or `function`

`"min"`/`"shorter"`, `"max"`/`"longer"`, `"length"`/`"fixed"`, `"ratio"`/`"relative"`, or `"none"`/`"full"`. A callback receives one dictionary with `base-length`, `length`, and `ratio`; the last two are positive absolute lengths or `none`. Unknown strings are rejected.

Default: `"min"`

segments

Return drawable cubic segments for any Kurvst path dictionary.

Parameters

```
segments(path: dictionary) -> array
```

path `dictionary`

Kurvst path dictionary to convert.

split-through

Split a path through a point sequence into per-span paths.

Parameters

```
split-through(
  points: array,
  omega: float,
  start-outset: int float,
  end-outset: int float,
  accuracy: float
) -> dictionary
```

points `array`

Two or more points for the curve to pass through.

omega `float`

Hobby curl/tension parameter.

Default: `1.0`

start-outset `int` or `float`

Arc length removed from the first span start.

Default: `0`

end-outset `int` or `float`

Arc length removed from the last span end.

Default: `0`

accuracy `float`

Geometry approximation accuracy passed to the Rust geometry engine.

Default: `0.001`

to-cetz

Draw a path dictionary through CeTZ.

Parameters

```
to-cetz(
  path: dictionary,
  unit: int float length ratio,
  ..style: any
) -> content
```

path `dictionary`

Kurvst path dictionary to draw.

unit `int` or `float` or `length` or `ratio`

Coordinate multiplier for emitted CeTZ points.

Default: 1

..style `any`

CeTZ draw style arguments forwarded to `merge-path`.

to-cetz-data

Emit a Kurvst path as CeTZ path data.

Parameters

```
to-cetz-data(
  path: dictionary,
  unit: int float length ratio
) -> array
```

path `dictionary`

Kurvst path dictionary to convert.

unit `int` or `float` or `length` or `ratio`

Coordinate multiplier for emitted CeTZ points.

Default: 1

to-native

Emit a Kurvst path as native Typst `curve` content.

Parameters

```
to-native(
  path: dictionary,
  unit: int float length ratio,
  ..style: any
) -> content
```

path `dictionary`

Kurvst path dictionary to emit.

unit `int` or `float` or `length` or `ratio`

Coordinate multiplier for emitted Typst curve points.

Default: 1

..style `any`

Native `curve` style arguments.

trim

Trim a path by arc length from each end.

Parameters

```
trim(
  path: dictionary,
  start-outset: int float,
  end-outset: int float,
  accuracy: float
) -> dictionary
```

path `dictionary`

Kurvst path dictionary to trim.

start-outset `int` or `float`

Arc length removed from the start.

Default: `0`

end-outset `int` or `float`

Arc length removed from the end.

Default: `0`

accuracy `float`

Arc-length approximation accuracy passed to the Rust geometry engine.

Default: `0.001`

wave

A smooth sinusoidal path pattern.

Parameters

```
wave(samples-per-period: int) -> dictionary
```

samples-per-period `int`

Number of samples used to approximate one wave period.

Default: `16`

zigzag

A straight-segment triangular path pattern.

Parameters

```
zigzag() -> dictionary
```

layer-defaults `dictionary`

Default geometry options for derived path layers.